

Project Proposal

Level 4

Intelligent Deployment Helper for Containerized Applications

DracaSys - Group no 24

Faculty of Information Technology

University of Moratuwa

2026

Project Proposal

Level 4

Intelligent Deployment Helper for Containerized Applications

DracaSys - Group no 24

Faculty of Information Technology

University of Moratuwa

2026

Aruran K	214020F
Jeyakumar T	214096P
Thanakaran R	214207K
Lehaananth V	214247G

Supervisor:

Dr. Firdhous M.F.M.

Table of Content

1. Introduction.....	4
2. Background & Motivation.....	4
3. Problem in Brief.....	4
4. Aim & Objectives.....	5
4.1 Aim.....	5
4.2 Objectives.....	5
5. Proposed Solution.....	6
Module 1: Trend & Seasonal Forecasting – Robust Hybrid Approach.....	6
Module 2: Short-Term Resource Prediction – Drift-Aware Approach.....	6
Module 3: System Anomaly Detector – Drift-Aware Approach.....	7
Module 4: Security Anomaly Detector – Drift-Aware Approach.....	7
6. Resource Requirements.....	8
7. References.....	9

1. Introduction

Containerized applications deployed on Docker and Kubernetes have become the backbone of modern cloud-native systems due to their scalability, portability, and efficiency. However, the dynamic and highly variable nature of these environments poses significant challenges, including sudden workload fluctuations, resource shortages, performance bottlenecks, and evolving security threats. Traditional monitoring and auto-scaling tools often react too late, leading to application downtime, degraded performance, and inefficient resource utilization.

To address these limitations, this project proposes the **Intelligent Deployment Helper**, an AI-driven system that continuously monitors containerized applications, accurately predicts resource demands across multiple time horizons, and detects both performance and security anomalies in real time. By integrating drift-aware forecasting and adaptive anomaly detection techniques, the system aims to enable proactive decision-making, reduce downtime, and optimize resource usage in dynamic Kubernetes-based environments.

2. Background & Motivation

Containerization technologies such as Docker and Kubernetes have become the de-facto standard for deploying modern cloud-native applications due to their lightweight nature, scalability, and portability. However, the dynamic and ephemeral characteristics of containerized environments introduce significant challenges in resource management and system reliability.

Modern container clusters experience rapid workload fluctuations, concept drift, bursty traffic patterns, and evolving security threats. Traditional rule-based auto-scaling and static monitoring tools often fail to respond effectively, resulting in application downtime, performance degradation, inefficient resource utilization, and increased security risks.

There is a growing need for an **intelligent, proactive system** that can continuously monitor containerized applications, accurately predict resource demands across multiple time horizons, detect both performance and security anomalies in real-time, and adapt to changing conditions. This project proposes an AI-driven **Intelligent Deployment Helper** that integrates forecasting, anomaly detection, and adaptive learning to address these challenges in dynamic Kubernetes-based environments.

3. Problem in Brief

Current container orchestration platforms suffer from the following critical limitations:

- Inaccurate short-term and long-term resource prediction under concept drift and bursty workloads.
- Inability to effectively detect complex performance bottlenecks involving multiple correlated metrics (CPU, memory, network, disk).
- Lack of robust drift-aware mechanisms in both performance and security anomaly detection, leading to model degradation over time.
- Poor handling of zero-day security attacks and multi-metric attack patterns in container environments.
- Absence of an integrated system that combines proactive forecasting with real-time anomaly detection for actionable decision support.

These issues result in frequent downtime, resource wastage, delayed scaling decisions, and increased vulnerability to both performance and security threats.

4. Aim & Objectives

4.1 Aim

The aim of this project is to design and develop an **integrated AI-driven Intelligent Deployment Helper** for containerized applications that proactively predicts resource usage, detects performance and security anomalies in real-time, and adapts to concept drift to prevent downtime, performance degradation, inefficient resource utilization, and security breaches.

4.2 Objectives

The main objectives of the project are:

1. To develop accurate short-term and long-term resource forecasting models capable of handling concept drift, bursty workloads, and missing data in containerized environments.
2. To design effective drift-aware anomaly detection mechanisms for identifying both performance bottlenecks and security threats using multivariate container metrics.
3. To implement online and incremental learning techniques that allow the system to continuously adapt to evolving workloads and attack patterns without frequent full retraining.
4. To integrate multi-metric correlation, adaptive thresholding, and attention mechanisms to improve prediction accuracy and anomaly detection reliability.
5. To evaluate the complete system using real-world container traces and synthetic workloads, measuring improvements in prediction accuracy, anomaly detection performance, drift adaptation speed, and overall system stability.
6. To demonstrate the practical applicability and effectiveness of the proposed intelligent helper in real container orchestration scenarios.

5. Proposed Solution

The **Intelligent Deployment Helper for Containerized Applications** is an integrated AI-driven system that continuously monitors Docker and Kubernetes environments. It predicts resource shortages across multiple time horizons and detects both performance and security anomalies in real time. To effectively handle **concept drift** a major challenge in dynamic containerized workloads the system employs four key adaptive techniques across modules: **adaptive sliding windows**, **online/incremental learning**, **error-based drift detection**, and **adaptive thresholding**.

Module 1: Trend & Seasonal Forecasting – Robust Hybrid Approach

Objective:

Predict long-term resource usage patterns, trends, daily/weekly seasonality, and gradual system growth in containerized environments to support capacity planning and proactive scaling decisions.

Core Technique:

Hybrid forecasting model combining Prophet (for trends and seasonality) and LSTM (for nonlinear residuals and spikes).

Key Novelties:

- Robust hybrid approach that effectively combines statistical and deep learning models for long-term forecasting.
- Peak-aware learning mechanism that prioritizes accuracy during critical high-resource usage periods.
- Missing-data robustness through simulation and interpolation techniques for reliable performance under imperfect real-world data.

Module 2: Short-Term Resource Prediction – Drift-Aware Approach

Objective: Forecast short-term resource usage (CPU, memory, network, and disk) in containerized environments to enable proactive scaling and prevent performance degradation under dynamic workloads.

Core Technique: GRU-based model designed for time-series forecasting of near-future resource demands.

Drift-Aware Mechanism:

- Adaptive sliding window (500–1000 recent samples) that adjusts based on workload variability.
- Online/incremental learning with error-triggered updates for continuous adaptation.
- Error monitoring using moving average and statistical checks to detect concept drift.
- Adaptive thresholding to dynamically adjust prediction confidence based on recent error patterns.

Key Novelties:

- Drift-aware short-term prediction combining adaptive sliding window, incremental learning, and error-based drift detection.
- Burst-aware forecasting to effectively handle sudden workload spikes.
- Multi-metric correlation modeling to capture interdependencies among CPU, memory, network, and disk usage.

Module 3: System Anomaly Detector – Drift-Aware Approach (Performance Anomalies)

Module 3 detects real-time performance-related anomalies such as CPU saturation, memory leaks, and network bottlenecks.

Core Technique: **Variational Autoencoder (VAE)** trained exclusively on normal (benign) data, combined with PCA for multi-metric correlation, using reconstruction error as the primary anomaly signal.

Drift-Aware Mechanism:

- Sliding window (30–60 timesteps) for focusing on recent system behavior with low memory usage.
- Online incremental learning via small SGD updates on recent data for continuous adaptation.
- Error distribution monitoring to detect concept drift by comparing current reconstruction errors with historical baseline.
- Adaptive thresholding based on recent error statistics ($\text{mean} + \alpha \times \text{std}$) to dynamically adjust anomaly decisions.

Key Novelties:

- Lightweight drift-aware anomaly detection by integrating sliding window, PCA, and a single VAE with incremental online learning.
- Efficient real-time adaptation to concept drift in containerized environments without full retraining or heavy ensembles.
- Effective multi-metric correlation using PCA and reconstruction error to detect CPU saturation, memory leaks, and network bottlenecks.
- Adaptive thresholding based on recent error statistics to reduce false positives and false negatives under varying workloads.

This design ensures lightweight and real-time deployable performance anomaly detection in dynamic environments.

Module 4: Security Anomaly Detector – Drift-Aware Approach

Objective: Real-time detection of malicious or suspicious behavior in containerized environments by analyzing CPU, memory, network flows, system calls, and file access logs.

Core Technique: **Attention-enhanced autoencoder** trained exclusively on normal (benign) data, using reconstruction error as the primary anomaly signal.

Drift-Aware Mechanism:

- Sliding window (500–1000 recent samples) for current behavior focus.
- Online/incremental learning for continuous model adaptation.
- Error monitoring with statistical tests (KS-test) on reconstruction error distribution.
- Adaptive thresholding to dynamically adjust anomaly decision boundary.

Key Novelties:

- True drift-aware detection combining sliding window, incremental learning, and error-based drift handling.
- Zero-day attack handling through high reconstruction error on unseen patterns.
- Multi-metric correlation monitoring via attention and joint error analysis to detect coordinated attacks across resource and security signals.

6. Resource Requirements**Hardware Requirements:**

- High-performance computing server or GPU-enabled machines (NVIDIA RTX 3060 / 4060 or equivalent) for model training.
- Minimum 32GB RAM and 500GB SSD storage for handling large container traces.

Software Requirements:

- Python 3.10+, PyTorch / TensorFlow, scikit-learn, Prophet, Pandas, NumPy.
- Kubernetes cluster (Minikube or kind) and Docker for testing.
- Prometheus + cAdvisor / Falco for data collection.
- Datasets: Alibaba Cluster Trace, CODEF, Sock Shop, synthetic fault-injection data.

7. References

- [1] Guruge, D., et al. (2025). Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and Long Short-Term Memory. *Frontiers in Computer Science*.
- [2] Priyadarshana, S., et al. (2025). Hybrid Prophet-LSTM approach for dynamic resource prediction and autoscaling in containerized environments. *Journal of Cloud Computing*.
- [3] Shao, Y., et al. (2026). Attention-based workload prediction and dynamic resource allocation in cloud-native environments. *IEEE Transactions on Cloud Computing*.
- [4] Li, J., et al. (2026). Drift-Aware Variational Autoencoder-based Anomaly Detection with Two-level Ensembling. arXiv:2602.12976.
- [5] Zhou, N., et al. (2026). An Abnormal File Access Detection Model for Containers Based on eBPF Listening. *Mathematics*.
- [6] Anemogiannisa, V., et al. (2025). Enhancing Kubernetes Resilience through Anomaly Detection and Prediction. arXiv:2503.14114.
- [7] Dkmak, G., et al. (2025). AI-Driven Anomaly Detection in Cloud-Native Microservices: The Night's Watch Algorithm. *Applied Sciences*.
- [8] Tunde-Onadele, O., et al. (2024). Self-Supervised Machine Learning Framework for Online Container Security Attack Detection. *ACM Transactions on Autonomous and Adaptive Systems*.
- [9] Ye, J., et al. (2025). U-SCAD: An Unsupervised Method of System Call-Driven Anomaly Detection for Containerized Edge Clouds. *Future Internet*.
- [10] Melnik, M., & Kotenko, I. (2025). Anomaly Detection in Containerized Systems Based on System Call Histograms and Autoencoder Neural Network. *EDM 2025*.
- [11] Kumar, R., et al. (2026). Praxium: Diagnosing Cloud Anomalies with AI-based Telemetry and Dependency Analysis. arXiv:2603.23890.
- [12] Zhou, C., et al. (2025). Concept Drift-Aware Anomaly Detection for Microservices in Kubernetes Using Temporal Autoencoders. *IEEE Transactions on Network and Service Management*.